

Programação Orientada a Objetos

-Trabalho Prático 2025/2026-

Meta 1



Artur Meireles de Sousa Falcão

-2024130485

Gabriel Cardoso Rodrigues

-2024133397-

Índice

Introdução -----	3
Decisões tomadas -----	3
Classes usadas na meta 1 -----	3
Descrição das classes -----	4
Relação das classes -----	5

Introdução

O trabalho prático proposto na disciplina de Programação Orientada a Objetos pretende criar um programa em C++ que representa a gestão de um **jardim virtual**, onde diversas entidades interagem entre si: tais como plantas, jardineiros, ferramentas e o próprio ambiente de simulação.

Para esta primeira meta do projeto foram pedidos os seguintes objetivos:

- a leitura de comandos e respetiva validação;
- o planeamento das classes;
- organização do projeto em ficheiros .h e .cpp.

Decisões tomadas

Durante o desenvolvimento da **Meta 1**, foram tomadas diversas decisões com o objetivo de simplificar e modularizar o projeto:

- Optou-se por uma **arquitetura modular**, separando a lógica do simulador (classe Simulador) da interface de visualização (classe Visualizador).
- As **posições e elementos do jardim** foram abstraídos em classes dedicadas (Posicao), facilitando a manipulação e extensão futura.
- Foi criada uma classe LeitorComandos responsável pela **leitura e execução dos comandos** do utilizador, garantindo maior separação de responsabilidades.
- As entidades principais Planta, Jardineiro e Ferramenta foram implementadas com **atributos específicos**, permitindo a evolução do simulador em metas seguintes.
- Utilizou-se **vetores e strings** como estrutura de dados base para armazenamento e gestão das entidades.

Classes usadas na Meta 1

- Simulador
- Visualizador
- Jardim
- Jardineiro
- Planta
- Ferramenta
- Posicao
- LeitorComandos

Descrição das classes

- **Simulador**

A classe Simulador é o **núcleo da aplicação**, responsável por gerir o estado global do jardim, coordenar as ações das restantes classes e processar os comandos introduzidos pelo utilizador.

Inclui métodos para iniciar o simulador, atualizar o estado do jardim e comunicar com o visualizador.

- **Visualizador**

A classe Visualizador trata da **representação visual do jardim**, imprimindo no ecrã a disposição atual das plantas, do jardineiro e das ferramentas.

Fornece uma interface simples para acompanhar a evolução da simulação.

- **Jardim**

A classe Jardim representa a **área de simulação**.

Contém as posições, elementos e a lógica associada à evolução do ambiente.

É responsável por armazenar e gerir as instâncias das classes Planta, Jardineiro e Ferramenta.

- **Jardineiro**

A classe Jardineiro representa o **agente ativo** do sistema (os cuidadores do jardim).

Cada jardineiro pode interagir com plantas, recolher ferramentas e executar ações de manutenção.

A classe define atributos como posição, estado e ações disponíveis.

- **Planta**

A classe Planta define os **atributos das plantas** no jardim, como tipo, estado de crescimento e posição.

Inclui métodos para simular o crescimento e interação com o ambiente (rega, corte, etc.).

- **Ferramenta**

A classe Ferramenta representa **utensílios usados pelo jardineiro**, como regadores ou tesouras.

É composta por métodos que simulam ações sobre as plantas ou o terreno e pode estar associada a um jardineiro específico.

- **Posicao**

Esta classe mostra os **pontos de localização** no jardim.

- **LeitorComandos**

A classe LeitorComandos é responsável por **interpretar e executar comandos** introduzidos pelo utilizador ou lidos de ficheiros de texto.

Permite controlar o simulador de forma textual e modular, traduzindo instruções em ações executadas pelo Simulador.

Relação das classes

- A classe **Simulador** relaciona-se diretamente com quase todas as outras, coordenando a execução do programa.
- **Jardim** agrega múltiplas instâncias de `Posicao`, que por sua vez contém referências a `Plantas` ou `Ferramentas`.
- **Jardineiro** interage com `Ferramenta` e `Planta`, modificando o estado do `Jardim`.
- **Visualizador** comunica com o `Simulador` para apresentar o estado atual da simulação.
- **LeitorComandos** envia instruções ao `Simulador`, que as distribui pelas restantes classes consoante a ação pedida.